

FakeBank Pro

Application bancaire mobile

Rapport d'analyse technique et fonctionnel

Application	FakeBank Pro
Technologie	Flutter / Dart
Backend	Supabase (PostgreSQL + Auth)
Etat global	Provider (ChangeNotifier)
APIs externes	OpenWeatherMap, Groq AI
Date du rapport	31/03/2026

1. Présentation générale

FakeBank Pro est une application mobile de banque fictive developpee avec Flutter. Elle simule un environnement bancaire complet permettant a plusieurs utilisateurs de s'inscrire, de gerer des comptes, d'effectuer des virements, et de consulter leur historique de transactions. L'application integre egalement un module meteo et un chatbot IA comme fonctionnalites secondaires.

1.1 Objectif de l'application

- Gestion multi-comptes par utilisateur avec creation libre
- Virements inter-utilisateurs en temps reel securises
- Authentification complete avec reinitialisation de mot de passe par OTP
- Widget meteo accessible sans authentication
- Chatbot IA capable d'interroger la meteo contextuelle

1.2 Stack technologique

Couche	Technologie	Role
Frontend	Flutter / Dart	Interface multiplateforme
Etat global	Provider	Gestion reactive des donnees
Backend / BDD	Supabase	Auth, PostgreSQL, RPC
Meteo	OpenWeatherMap API	Donnees meteorologiques temps reel
IA Chatbot	Groq API	Reponses conversationnelles LLM

2. Architecture

2.1 Structure des fichiers

Dossier	Contenu
screens/	Toutes les vues UI (login, home, comptes, virements...)
providers/	AuthProvider et BankProvider - logique metier reactive
core/	SupabaseService - couche d'accès aux données
models/	AccountModel, TransactionModel, UserModel
widgets/	Composants réutilisables (TransactionTile, BalanceCard)
meteo.dart	Module meteo + chatbot IA (fichier autonome)

2.2 Flux de données

Le pattern Provider centralise la gestion d'état. Le flux typique d'une action suit ces étapes :

- L'utilisateur interagit avec un Screen (ex. : saisir un virement)
- Le Screen appelle le Provider correspondant (BankProvider.transfer())
- Le Provider délègue à SupabaseService qui exécute la requête
- Le Provider met à jour son état et appelle notifyListeners()
- Tous les widgets abonnés via context.watch se reconstruisent automatiquement

2.3 Providers

AuthProvider

Gère le cycle de vie de l'authentification : connexion, inscription, déconnexion, et réinitialisation du mot de passe en 3 étapes (email -> OTP -> nouveau mot de passe). Expose : user, isAuthenticated, isLoading, error, resetEmail.

BankProvider

Gère les comptes et transactions de l'utilisateur connecté. Fournit loadAll(), createAccount(), transfer(), searchUsers(). Met à jour le solde localement après un virement sans rechargement complet.

3. Fonctionnalités détaillées

3.1 Authentification

Fonctionnalité	Description
Inscription	Prenom, nom, email, mot de passe (min. 6 car.), solde initial
Connexion	Email + mot de passe avec erreurs localisées en français
Déconnexion	Efface les données locales et redirige vers login
Mot de passe oublié	Flux 3 étapes : email -> OTP 8 chiffres -> nouveau mot de passe
Restauration session	Vérification de session active au démarrage

3.2 Gestion des comptes

A l'inscription, un compte principal est cree automatiquement avec le solde initial choisi. L'utilisateur peut creer des comptes supplementaires depuis l'onglet Comptes.

- Chaque compte possede un numero au format FR76-XXXXXXXX (genere automatiquement)
- L'ecran de detail affiche le solde, le proprietaire, et l'historique lie
- Un bouton de virement rapide est accessible depuis le detail du compte

3.3 Virements

- Selection du compte source avec affichage du solde disponible
- Recherche du destinataire par nom, prenom ou email (min. 2 caracteres)
- Execution via procedure RPC Supabase (do_transfer) garantissant l'atomicite
- Mise a jour locale immediate du solde apres confirmation
- Champ motif facultatif (60 caracteres max)

3.4 Historique des transactions

L'onglet Historique affiche toutes les transactions impliquant les comptes de l'utilisateur, trieés de la plus recente a la plus ancienne. Chaque ligne indique l'emetteur, le destinataire, le montant (rouge si debit, vert si credit), la date/heure, et le motif.

3.5 Module Meteo et Chatbot IA

Accessible depuis la page de connexion sans authentication, le module meteo propose :

- WeatherPage : recherche d'une ville, affichage temperature, description, icone via OpenWeatherMap
- ChatbotPage : interface de chat alimentee par l'API Groq, capable de repondre et de recuperer automatiquement la meteo d'une ville mentionnee
- Historique de conversation persiste via Supabase
- Support des themes clair et sombre

4. Modèles de données

4.1 UserModel

Champ	Type	Description
id	String (UUID)	Identifiant unique Supabase Auth
firstName	String	Prenom de l'utilisateur
lastName	String	Nom de famille
email	String	Adresse email (unique)

4.2 AccountModel

Champ	Type	Description
id	String (UUID)	Identifiant unique du compte
accountNumber	String	Numero format FR76-XXXXXXXX
label	String	Libelle (ex. Compte courant)
balance	double	Solde courant en euros
ownerId	String	Reference vers l'utilisateur proprietaire
ownerName	String	Nom complet (jointure SQL)

4.3 TransactionModel

Champ	Type	Description
id	String (UUID)	Identifiant de la transaction
amount	double	Montant en euros
fromAccountId / toAccountId	String	Comptes source et destination
fromOwnerName / toOwnerName	String	Noms des proprietaires
note	String?	Motif facultatif du virement
createdAt	DateTime	Horodatage de la transaction

5. Sécurité

5.1 Points positifs

- Authentification geree par Supabase Auth (JWT) - mots de passe jamais stockes en clair
- Reinitialisation via OTP a usage unique envoye par email
- Transactions atomiques via procedure stockee RPC
- Validation des montants cote client (> 0, format numerique valide)
- Messages d'erreur d'authentification localises et explicites

5.2 Points d'attention

- La cle anonyme Supabase (anonKey) est exposee en clair dans main.dart et supabase_config.dart - a externaliser en variable d'environnement en production
- La cle OpenWeatherMap est codee en dur dans meteo.dart
- La cle Groq est injectee via String.fromEnvironment() - correct mais a verifier dans le pipeline CI/CD
- Absence de rate limiting cote client sur les tentatives de connexion
- La recherche utilisateurs expose les comptes de tous les utilisateurs inscrits

6. Interface utilisateur

6.1 Design system

L'application adopte un design system coherent base sur Material Design 3 avec une palette bleu bancaire (0xFF1565C0 a 0xFF1E88E5). Les degradés lineaires sont utilises systematiquement sur les AppBar, les fonds d'ecran et les cartes de comptes.

6.2 Navigation

La navigation principale post-connexion repose sur une barre NavigationBar avec trois onglets : Comptes, Virement, Historique. Un IndexedStack preserve l'etat de chaque onglet lors du changement.

6.3 Liste des écrans

Ecran	Role
LoginScreen	Connexion, acces inscription et meteo
RegisterScreen	Creation de compte avec solde initial
HomeScreen	Shell principal avec navigation par onglets
AccountsScreen	Liste comptes + creation nouveau compte
AccountDetailScreen	Detail compte + historique + virement rapide
CreateAccountScreen	Formulaire de creation de compte
TransferScreen	Formulaire de virement avec recherche
ForgotPasswordScreen	Saisie email reinitialisation
OtpVerifyScreen	Verification code OTP 8 chiffres
NewPasswordScreen	Saisie du nouveau mot de passe
WeatherPage	Recherche meteo par ville
ChatbotPage	Chatbot IA avec contexte meteo

7. Intégrations externes

7.1 Supabase

- Auth : signUp, signInWithPassword, signOut, signInWithOtp, verifyOTP, updateUser
- Base de donnees PostgreSQL : tables profiles, accounts, transactions
- Procedure stockee : do_transfer pour garantir l'atomicite des virements

7.2 OpenWeatherMap

Appel HTTP GET pour recuperer les conditions meteo d'une ville. Donnees : temperature, description, humidite, vent, icone. Cle API stockee en dur dans meteo.dart.

7.3 Groq AI

API compatible OpenAI pour alimenter le chatbot. Le modele recoit un historique de conversation et un contexte meteo enrichi. Cle injectee via --dart-define=GROQ_API_KEY.

8. Points forts et axes d'amélioration

8.1 Points forts

- Architecture propre et bien separee (screens / providers / services / models)
- Gestion d'erreurs robuste avec messages localises en francais
- Virements atomiques via procedure stockee Supabase
- Flux complet de reinitialisation de mot de passe par OTP
- Chatbot IA avec enrichissement meteo automatique
- Design coherent et moderne (Material 3, degradés bleu bancaire)
- Mise a jour locale immediate du solde apres virement (UX reactive)

8.2 Axes d'amélioration suggérés

- Externaliser les clés API dans des variables d'environnement
- Ajouter des tests unitaires et de widget
- Implementer une pagination sur l'historique des transactions
- Ajouter une confirmation avant tout virement
- Securiser la recherche utilisateurs (confidentialite des comptes)
- Internationalisation (i18n) pour adapter a d'autres langues
- Mode hors-ligne avec cache local des donnees recentes

9. Conclusion

FakeBank Pro est une application Flutter bien structuree qui couvre un perimetre fonctionnel ambitieux : authentication complete, gestion multi-comptes, virements securises, et fonctionnalites bonus (meteo, chatbot IA). L'architecture respecte les bonnes pratiques Flutter avec une separation claire des responsabilites et une gestion d'etat reactive via Provider.

Les principaux risques identifies concernent la securite des clés API exposees en clair et l'absence de tests automatises. Ces points sont a traiter en priorite avant tout deploiement en production. Pour un usage pedagogique ou de demonstration, l'application constitue un excellent exemple d'application Flutter full-stack.